

# Connected Vehicle Intelligence Hackathon

## Solution Document: FleetPulse

**Submission Format:** PDF (export of this document)

**To be Submitted by:** [Team Name]

**Team Members & Roles:** [Name - role - email, for each member]

**Problem Space Chosen:** Predictive maintenance and real-time fault intelligence for multi-tenant commercial fleets (100,000 connected vehicles, three OEM clouds)

**Repository URL:** <https://github.com/mayukhp24/motorq-hackathon>

**Demo Video URL ( $\leq 5$  min):** [Link]

**Date of Submission:** [DD/MM/YYYY]

# Table of Contents

- [1. Executive Summary](#)
- [2. Problem Statement & Validation](#)
  - [2.1 Problem Statement](#)
  - [2.2 Evidence & Validation](#)
  - [2.3 Impact & Success Metrics](#)
- [3. Solution Description](#)
  - [3.1 Solution Overview & User Journey](#)
  - [3.2 Key Value Proposition](#)
  - [3.3 Innovative Ideas](#)
- [4. Feature List](#)
- [5. Solution Architecture \(High-Level Design\)](#)
  - [5.1 Architecture Overview](#)
  - [5.2 Technology Stack & Justification](#)
  - [5.3 Data Architecture](#)
  - [5.4 Deployment View](#)
- [6. Low-Level Design](#)
  - [6.1 Layering & Separation of Concerns](#)
  - [6.2 Design Principles Applied](#)
  - [6.3 Design Patterns Used](#)
  - [6.4 Interfaces, Contracts & Runtime Flows](#)
  - [6.5 Algorithms & Data Structures](#)
- [7. Non-Functional Requirements & Performance Benchmarks](#)
- [8. Security & Compliance](#)
- [9. Test Strategy](#)
- [10. Observability](#)
- [11. AI / ML Component \(if used\)](#)
- [12. Architecture Decisions, Risks & Future Enhancements](#)
- [13. Demo Video \(5 Minutes Maximum\)](#)
- [14. Repository Checklist](#)
- [15. Conclusion](#)
- [16. Declarations](#)
- [17. Appendix \(if any\)](#)

# 1. Executive Summary

Brief overview (half a page maximum) of:

- The problem you chose and who it affects
- What your solution does, in one or two sentences
- Key results achieved (throughput, latency, impact numbers)
- Unique features, innovations or optimisations you implemented

**Problem.** A roadside breakdown costs a commercial fleet about \$2,400 (tow, emergency repair, lost day) against about \$520 for the same repair planned in the workshop. Maintenance managers today get either a flood of raw fault codes or fixed-threshold alerts that miss slow degradations, and every OEM cloud reports in a different format. Three competing fleet operators (100,000 vehicles in total) need this on one platform without seeing each other's data.

**Solution.** FleetPulse ingests telemetry from 100,000 vehicles across three OEM formats, raises 12 kinds of critical fault in real time, and ranks every vehicle by its probability of breaking down in the next 7 days, per component, with the reason and the dollar value of acting now, plus a maintenance copilot that answers from live data and proposes work orders for a manager to approve.

## Key results (measured).

- Critical alert visible via the API **0.02-0.54 s** after the OEM sends it; p95 **480 ms** (target < 5 s).
- API at 100 req/s for 5 min: p95 **135 ms**, p99 **408 ms**, **0 errors** in 30,025 requests (targets 200 / 500 ms).
- Prediction on a held-out later period: precision **0.91 vs 0.41** for today's threshold rules at the same workshop capacity, PR-AUC 0.66 vs 0.21, net savings **\$8.3M vs \$4.3M**.
- Pipeline stages benchmarked at ~120K events/s per core (processor) and ~100K records/s per core (gateway); 100K events/s is sized from these with horizontal scaling (the 4-vCPU build machine sustained ~4K events/s end to end).
- **5/5 chaos experiments** pass (processor kill, Kafka outage, ClickHouse outage, Redis wipe); query optimisations of up to 400× (e.g. 351 ms → 1.3 ms, 2,911 ms → 7.3 ms).

**What is distinctive.** Declarative OEM adapters (a new format is a spec, not code); rotating Bloom-filter dedup plus deterministic IDs for effectively-once results over at-least-once transport; defence-in-depth tenant isolation (JWT → row-level security → ClickHouse row policy); calibrated per-component risk with plain-language reasons; a provider-agnostic copilot (free or self-hosted models work) whose tools run under the user's permissions and can only propose actions; a hash-chained audit log and a right-to-erasure API.

## 2. Problem Statement & Validation

### 2.1 Problem Statement

- **Format:** “[User] needs a way to [job to be done] because [pain], which today costs [measurable impact].”
- **Primary User:** Fleet manager, dealer, lender, driver or OEM — name one
- **Secondary Stakeholders:** Others affected by the problem

**Fleet maintenance managers need a way to** know which vehicles to pull into the workshop this week, and which faults need action now, **because** raw OEM telemetry arrives in incompatible formats and today's fixed-threshold alerts either flood them with noise or miss slow degradations, **which today costs** about \$1,900 per avoidable roadside breakdown (unplanned  $\approx$  \$2,400 vs planned  $\approx$  \$520 for a cooling failure; \$7,800 vs \$2,600 for an EV traction pack), plus unsafe roadside stops and lost delivery days.

**Primary user:** the fleet maintenance manager (maintenance lead) of a commercial fleet operator.

**Secondary stakeholders:** drivers (safety, fewer breakdowns), fleet operations and finance (uptime, idle cost), fleet administrators and compliance officers (access control, audit, data-subject requests), OEMs and dealers (warranty and parts demand), and the platform operator running many tenants.

### 2.2 Evidence & Validation

- **Facts vs Assumptions:** Separate what you know from what you assume
- **Validation Method:** Published study, public dataset, simulation, or interviews
- **Existing Alternatives:** Aftermarket dongles, OEM portals, Motorq Fuse, other telematics tools — and why they fall short

| Evidence / Assumption                                                                                  | Source or Method                                                                              | What It Shows                                                     | Confidence                      |
|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------------|---------------------------------|
| Unplanned breakdowns cost several times a planned repair (tow, emergency labour, downtime)             | Industry repair-cost ranges; FleetPulse cost model per component (configurable)               | $\approx$ 4-5 $\times$ cost multiplier: the value of acting early | Medium (assumption)             |
| Precursor signals exist days before a failure (coolant creep, charging-voltage sag, rising DTC counts) | Encoded in the simulator's component-health model; consistent with OBD-II diagnostic practice | A 7-day lead time is learnable                                    | Medium (validate on pilot data) |
| Fixed-threshold rules have low precision                                                               | Back-test on a held-out later period (638,358 vehicle-days)                                   | Rules: precision 0.41 at 1% capacity; model 0.91                  | High on our data                |

| Evidence / Assumption                                                 | Source or Method                                                                                      | What It Shows                                                      | Confidence  |
|-----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|-------------|
| OEM clouds differ in structure, units and time formats                | Problem brief; implemented for 3 formats                                                              | Normalisation layer is mandatory                                   | High (fact) |
| Idling wastes fuel                                                    | Published idle fuel-use estimates ( $\approx 0.6\text{--}1.9$ L/h for light vehicles); we use 0.9 L/h | Idle hours $\times$ fuel price is a visible, actionable cost       | Medium      |
| Competing fleets require strict isolation and minimised personal data | GDPR / DPDP Act 2023 obligations                                                                      | Tenant isolation and privacy are product requirements, not add-ons | High (fact) |

**Facts vs assumptions.** Facts: the brief's scale and NFRs, OEM format diversity, regulatory duties, and everything measured on our running system. Assumptions: repair costs and precursor behaviour are modelled (documented in `services/analytics/analytics/features.py` and the simulator) and must be validated with a pilot fleet's history.

**Validation method.** Simulation plus back-test: a deterministic simulator generates 60 days of history for 100,000 vehicles with hidden component degradation, noise, duplicates, out-of-order data and outages; the model is trained on the earlier period and evaluated on the later one against the rules fleets use today, at the same workshop budget.

**Existing alternatives and gaps.** Aftermarket OBD dongles: hardware cost and installation, limited signals, no cross-fleet model. OEM portals: one brand per portal, no unified view or tenant-level prioritisation. Connected-car data platforms such as Motorq Fuse: normalise and deliver OEM data; FleetPulse is the decision layer on top (detection, prediction, workflow). Generic telematics tools: rule-based alerts with the precision problem shown above.

## 2.3 Impact & Success Metrics

| Metric                                                       | Baseline Today                             | Target                 | How Measured / Estimated                                            |
|--------------------------------------------------------------|--------------------------------------------|------------------------|---------------------------------------------------------------------|
| Precision of maintenance flags at a 1% daily workshop budget | 0.41 (threshold rules)                     | $\geq 0.80$            | Back-test on held-out period: <b>0.91 achieved</b>                  |
| Share of 7-day breakdowns flagged at that budget (recall)    | 0.21                                       | $\geq 0.40$            | Same back-test: <b>0.45</b>                                         |
| Net savings, 100K vehicles, 15-day test window               | \$4.3M (rules)                             | $\geq 1.5\times$ rules | Avoided cost – planned repair – \$80 per false alarm: <b>\$8.3M</b> |
| Time from fault to manager's screen                          | Portal polling / e-mail (minutes to hours) | $< 5$ s                | Measured: 0.02–0.54 s, p95 480 ms                                   |

| Metric               | Baseline Today | Target              | How Measured / Estimated                         |
|----------------------|----------------|---------------------|--------------------------------------------------|
|                      | assumption)    |                     |                                                  |
| Idle cost visibility | None / manual  | Per vehicle per day | ClickHouse idle-cost report (Cost & safety page) |

- **Scale of Impact:** Who benefits, by how much, at 10K vs 100K vehicles
- **Wider Impact:** Safety, cost, environment or compliance, where relevant

**Scale of impact.** The synthetic back-test roughly doubles avoided cost versus rules. Real savings depend on a fleet's breakdown rate; at a conservative 1 avoidable breakdown per 10 vehicles per year and \$1,880 avoided per event, capturing 45% of them saves **≈ \$0.85M/year for a 10K-vehicle fleet** and **≈ \$8.5M/year at 100K** (false-alarm cost is small at 0.91 precision). Platform cost grows sub-linearly: stateless services scale horizontally and storage is ~31 bytes per telemetry row.

**Wider impact.** Safety: overheating, crash, low-voltage and risky-driving alerts reach the manager in under a second, and fewer vehicles break down on the roadside. Environment: idle time is priced and ranked (a 30% idle reduction is estimated per fleet in the Cost & safety view; every litre of diesel avoided is ≈ 2.7 kg CO<sub>2</sub>). Compliance: tenant isolation, minimisation, audit and erasure are built in (GDPR, DPDP Act 2023).

## 3. Solution Description

### 3.1 Solution Overview & User Journey

- **What It Does:** Plain-language description of the solution
- **User Journey:** Step by step, e.g. vehicle event → detection → alert → action → outcome
- **Screenshot:** At least one screenshot of the working product

**What it does.** FleetPulse is a web platform for fleet operators. It collects every vehicle's telemetry from the manufacturers' clouds, watches it continuously for faults that need action now, predicts which vehicles are likely to break down in the coming week and why, and helps the maintenance team act: schedule the repair, track the work order, and ask questions in plain language.

**User journey: real-time fault.**

1. A delivery van's coolant climbs to 118 °C; its OEM cloud batches the readings to FleetPulse over MQTT or HTTPS.
2. The ingest gateway decodes the OEM format, validates VIN and ranges, and writes canonical events to Kafka.

3. The stream processor sees the temperature held above 110 °C for 15 s and raises a CRITICAL ENGINE\_OVERHEAT alert (duplicates and late readings cannot re-trigger it).
4. Within a second the alert appears on the maintenance manager's Overview (WebSocket) and in Alerts.
5. The manager acknowledges it, calls the driver to stop, and schedules a repair; the audit log records every step.

### User journey: predicted breakdown.

1. Nightly, features are computed per vehicle-day and the model scores all 100,000 vehicles.
2. Monday morning, Predictive maintenance lists the riskiest vehicles with the likely failing component, the reasons ("2 HV battery fault codes in 3 days; pack temperature 44.5 °C") and the saving from acting now.
3. The manager clicks Schedule (or asks the copilot to "schedule a repair for the riskiest one" and approves its proposal); the vehicle goes to the workshop instead of breaking down on the road.

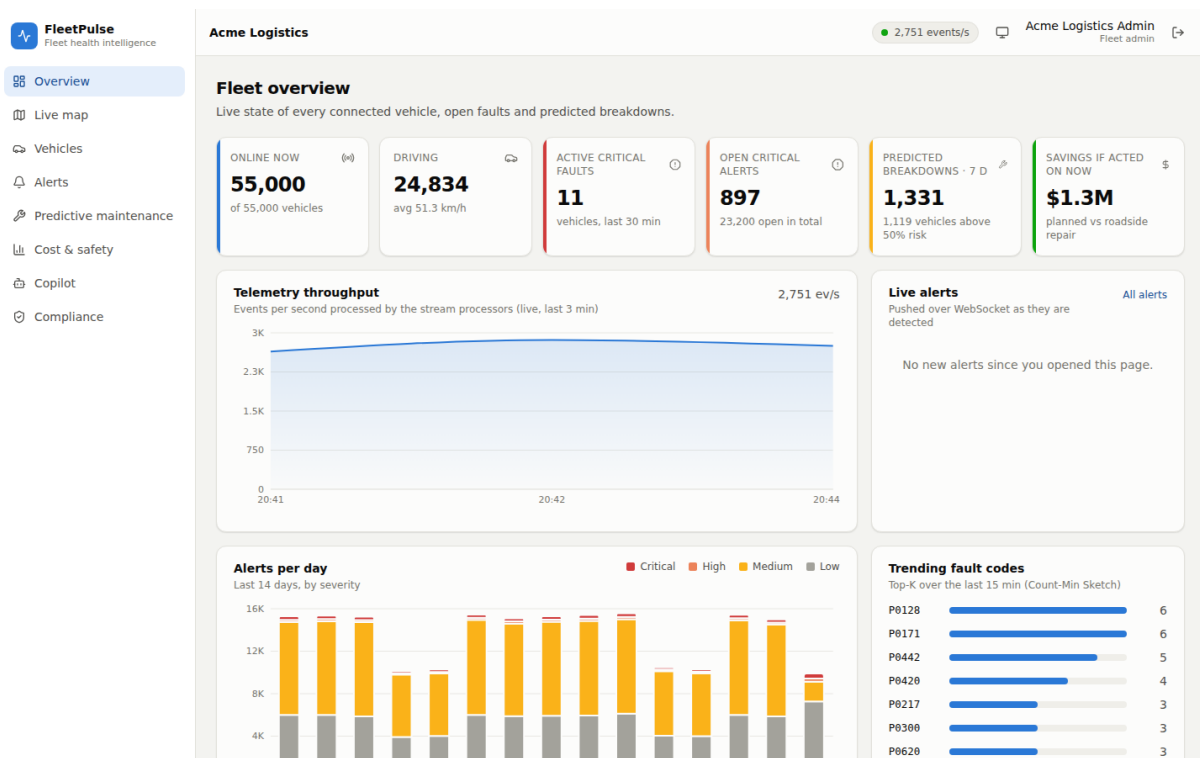


Figure 1. Fleet overview: live KPIs, throughput, alerts pushed over WebSocket, alert trend and trending fault codes.

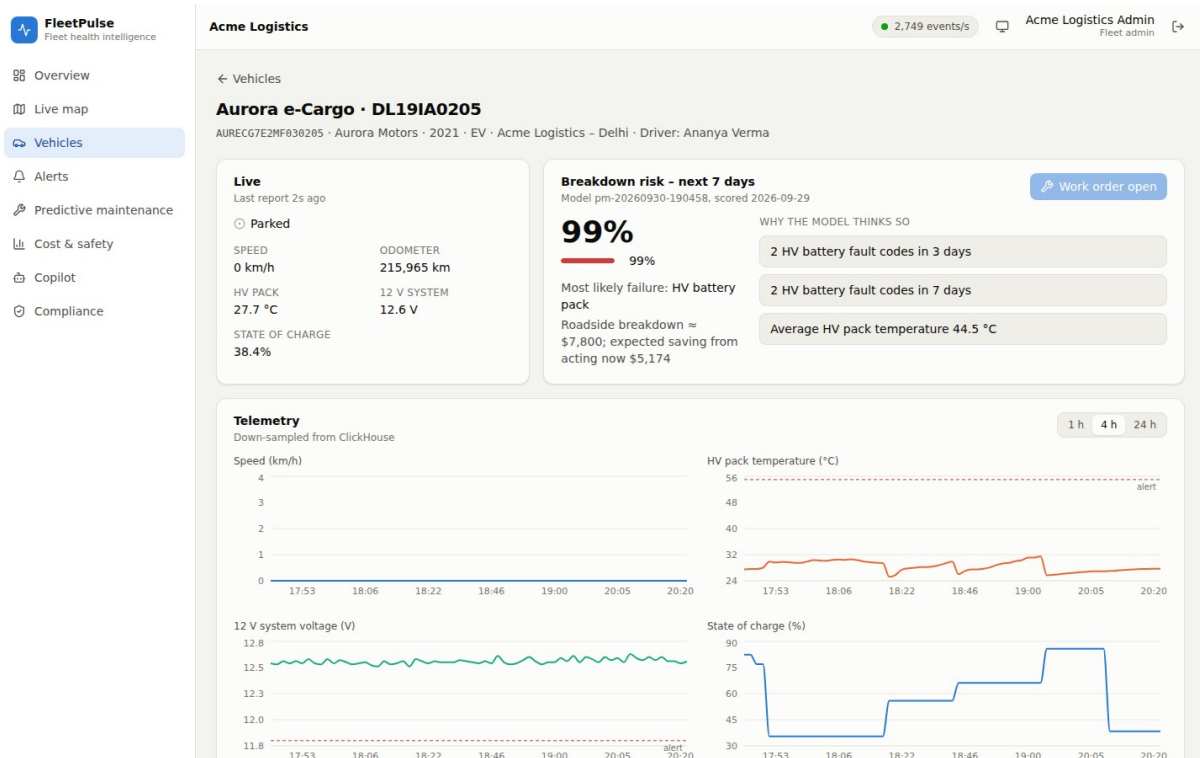


Figure 2. Vehicle detail: live state, 7-day breakdown risk with the model's reasons and dollar impact, telemetry history.

### 3.2 Key Value Proposition

- **Customer Job:** What the user is trying to get done
- **Pain Relieved:** What gets easier, cheaper or safer
- **Gain Created:** New capability the user did not have
- **Differentiation:** Why your solution, not the alternatives

**Customer job:** keep every vehicle on the road at the lowest maintenance cost, and react immediately when something dangerous happens.

**Pain relieved:** no more reading raw fault codes across three OEM portals; far fewer false alarms (precision 0.91 vs 0.41); critical faults surface in under a second instead of via portal polling.

**Gain created:** a ranked, explained, priced list of which vehicles to service this week; one view across all OEMs; cost-of-idling and driver-safety insight; a copilot that turns questions into answers and approved actions.

**Differentiation:** built for multiple competing tenants from the start (database-enforced isolation, privacy by role), explainable predictions benchmarked against today's rules, and an AI assistant that cannot act without human approval.

### 3.3 Innovative Ideas

- **Up to Three Ideas:** A technique, data insight, user experience or business model
- **What Is New:** Compared against prior work you studied
- **Evidence:** Proof that each idea works

#### 1. Declarative, hot-loadable OEM adapters with contracts checked in CI.

Each OEM format is a JSON spec (field paths, units, time format) compiled once into a fast decoder; the canonical event and all Kafka topics have JSON Schemas verified from both producer and consumer sides. **New:** most integrations hand-code a parser per OEM; here onboarding a format is a data change. **Evidence:** TestHotOnboardNewOEM, contract tests validating 3,000 telemetry events, 689 alerts and 321 trips per run; malformed records go to a DLQ while the rest of the batch is accepted (BDD scenario).

**2. Explainable, calibrated per-component risk, measured against today's rules.** One gradient-boosted model per failure mode, combined with noisy-OR, prior-corrected back to true probabilities, with reasons rendered as sentences and a dollar impact. **New:** evaluation at equal workshop capacity against the threshold rules fleets actually use, not only AUC. **Evidence:** precision 0.91 vs 0.41, top-decile calibration 14.8% predicted vs 15.0% observed, time-split with leakage tests.

**3. A copilot that can only do what the user can, and only propose.** Tools are filtered by the user's permissions and execute under row-level security; write actions become proposals that a manager approves; VINs in answers must be grounded in tool results; a deterministic planner keeps it working without the LLM. **Evidence:** integration and BDD tests (viewer cannot approve; injection refused; one pending proposal per vehicle), every tool call in the tamper-evident audit log.

## 4. Feature List

Every feature must be traceable to code. Priority uses MoSCoW (Must / Should / Could / Won't). Status: Done / Partial / Planned. Give the demo video timestamp for Done features.

| ID   | Feature                                       | User Story                                                                                                     | Priority | Status | Code Path                                                  | Video |
|------|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------|----------|--------|------------------------------------------------------------|-------|
| F-01 | Multi-OEM ingestion (MQTT + HTTPS, 3 formats) | As a platform operator I want every OEM's data in one canonical form so that all features work for every brand | Must     | Done   | services/<br>pipeline/<br>internal/ingest,<br>internal/oem | 1:05  |
| F-02 | Validation, DLQ and back-pressure             | As a data engineer I want bad records isolated without losing good ones so that data stays trustworthy         | Must     | Done   | internal/<br>canonical,<br>ingest/gateway.go               | 1:10  |
| F-03 | Real-time critical                            | As a maintenance                                                                                               | Must     | Done   | internal/                                                  | 1:15  |

| ID   | Feature                                            | User Story                                                                                            | Priority | Status | Code Path                                             | Video |
|------|----------------------------------------------------|-------------------------------------------------------------------------------------------------------|----------|--------|-------------------------------------------------------|-------|
|      | alerts (12 rules)                                  | manager I want overheating and crashes flagged within seconds so that I can stop the vehicle          |          |        | detect, internal/processor                            |       |
| F-04 | Live overview and KPIs over WebSocket              | As a manager I want a live picture of my fleet so that I see problems as they happen                  | Must     | Done   | web/src/pages/Overview.tsx, api/fleet.py              | 1:00  |
| F-05 | Live map with density clusters                     | As a dispatcher I want to see where vehicles are so that I can plan recovery                          | Should   | Done   | web/src/pages/LiveMap.tsx, internal/stream/geohash.go | 1:45  |
| F-06 | Vehicle 360 (live, history, risk, alerts, service) | As a manager I want one page per vehicle so that I can decide quickly                                 | Must     | Done   | web/src/pages/VehicleDetail.tsx, services/fleet.py    | 1:30  |
| F-07 | 7-day breakdown prediction with reasons and \$     | As a manager I want the vehicles most likely to fail and why so that I can service them first         | Must     | Done   | services/analytics/analytics/model.py, jobs.py        | 2:00  |
| F-08 | Work orders from predictions                       | As a manager I want to schedule a repair in one click so that nothing falls through                   | Must     | Done   | services/maintenance.py, pages/Maintenance.tsx        | 2:15  |
| F-09 | Maintenance copilot with human approval            | As a manager I want to ask questions in plain language so that I get answers without building reports | Should   | Done   | fleet-api/app/agent/                                  | 2:20  |
| F-10 | Idle cost and driver safety analytics              | As an operations lead I want idling cost and risky drivers so that I can cut waste and coach          | Should   | Done   | services/analytics.py, db read models                 | 2:40  |
| F-11 | Tenant isolation, RBAC, location masking           | As a fleet admin I want competitors never to see my data and staff to see only what they need         | Must     | Done   | security/, db/postgres/migrations/003_*               | 2:47  |
| F-12 | Tamper-evident audit and right to erasure          | As a compliance officer I want proof of access and erasure on request so that we meet GDPR/DPDP       | Must     | Done   | services/compliance.py, pages/Compliance.tsx          | 3:00  |
| F-13 | 100K-vehicle simulator with                        | As an evaluator I want realistic load so that                                                         | Must     | Done   | services/pipeline/                                    | 3:12  |

| ID   | Feature                                           | User Story                                                                                 | Pri<br>ori<br>ty   | Statu<br>s                                         | Code Path                                         | Vide<br>o |
|------|---------------------------------------------------|--------------------------------------------------------------------------------------------|--------------------|----------------------------------------------------|---------------------------------------------------|-----------|
|      | bursts,<br>duplicates,<br>outages                 | claims are tested                                                                          |                    |                                                    | internal/sim                                      |           |
| F-14 | Observability dashboard and alerts                | As an SRE I want throughput, lag and latency visible so that I can act before users notice | Must               | Done                                               | deploy/<br>observability                          | 3:35      |
| F-15 | Failure recovery (processor, broker, stores)      | As an SRE I want the system to recover without data loss                                   | Must               | Done                                               | tests/chaos/<br>chaos.py                          | 3:42      |
| F-16 | Platform operator view (tenants, data quality)    | As the platform operator I want tenant health and ingestion quality                        | Should             | Done                                               | web/src/<br>pages/<br>Platform.tsx,<br>api/ops.py | —         |
| F-17 | Cloud deployment (Helm, Terraform AWS/GCP values) | As an operator I want a repeatable cloud deployment                                        | Must               | Partial (validated, not applied to a live account) | deploy/helm,<br>infra/terraform<br>/aws           | 4:05      |
| F-18 | External OIDC identity federation                 | As an enterprise admin I want SSO with our IdP                                             | Could              | Planned                                            | app/security/<br>jwt.py                           | —         |
| F-19 | EV charging optimisation                          | As an EV fleet manager I want charging scheduled around routes and tariffs                 | Won't (this phase) | Planned                                            | —                                                 | —         |

## 5. Solution Architecture (High-Level Design)

### 5.1 Architecture Overview

- **System Context Diagram:** C4 level 1 — your system, users and external systems (OEM clouds, identity provider, LLM API, data warehouse)
- **Container Diagram:** C4 level 2 — every service, data store and broker, with the protocol on each arrow (MQTT, HTTPS, gRPC, Kafka)
- **Data Flow:** Path of one telemetry event from vehicle to insight, with expected latency per hop

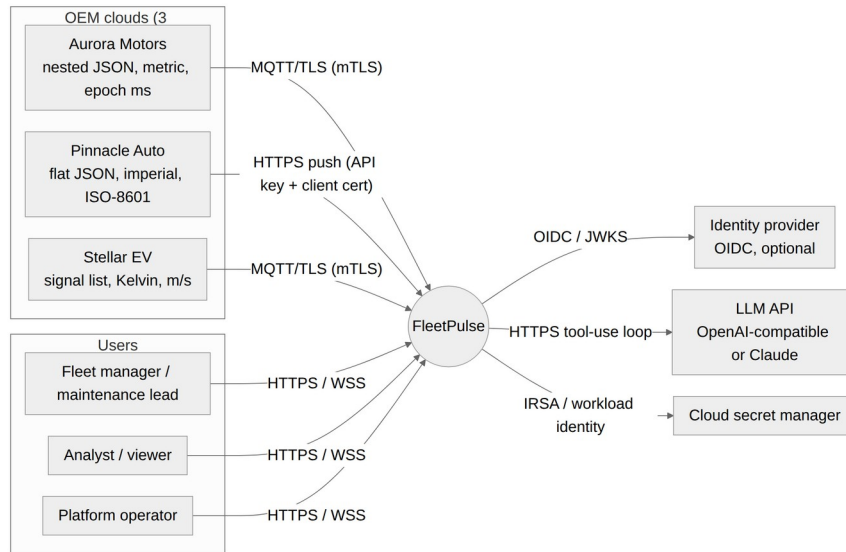


Figure 3. System context (C4 level 1).

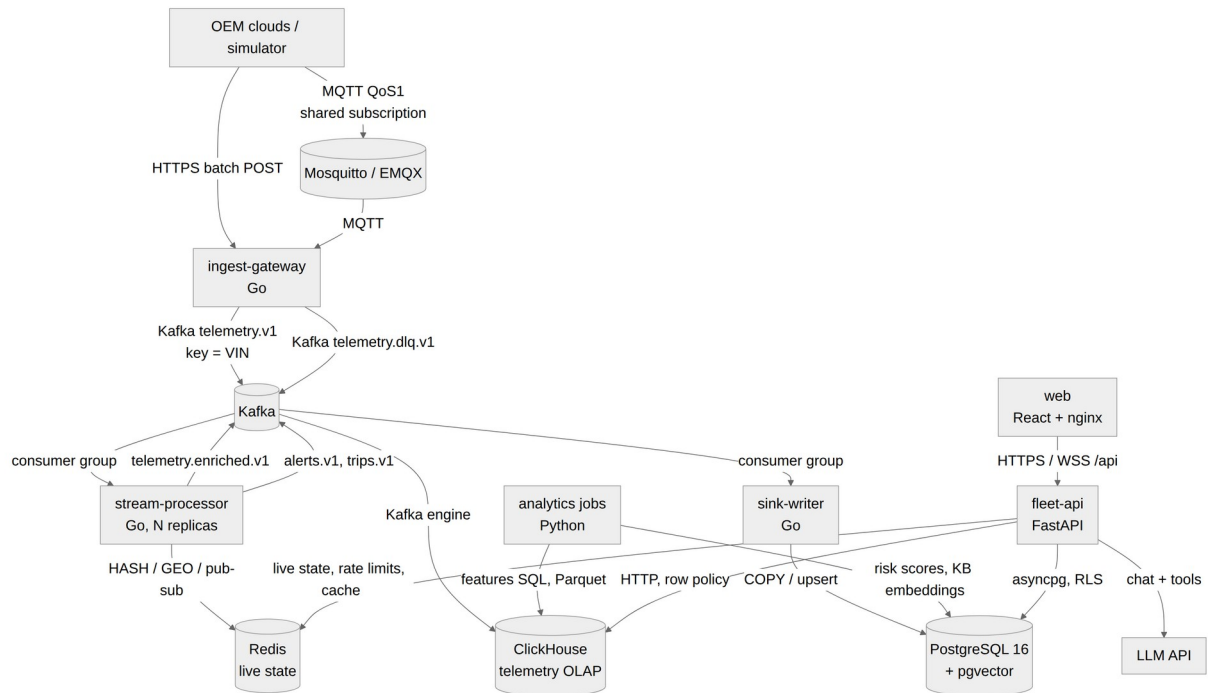


Figure 4. Containers (C4 level 2), with the protocol on each arrow.

**Data flow: one telemetry event from vehicle to insight** (measured on the compose stack with 100K vehicles at ~4K events/s; per-record costs from single-core benchmarks):

| # | Hop                                   | Mechanism                                            | Latency       |
|---|---------------------------------------|------------------------------------------------------|---------------|
| 1 | Vehicle → OEM cloud → FleetPulse edge | OEM batches ≤ 500 records over MQTT QoS 1 or HTTPS   | network-bound |
| 2 | Gateway decode + validate             | Adapter per OEM, VIN check digit, ranges, clock skew | ~10 μs/record |

| # | Hop                      | Mechanism                                                     | Latency                            |
|---|--------------------------|---------------------------------------------------------------|------------------------------------|
| 3 | Gateway → Kafka          | Idempotent producer, acks=all, lz4, linger 5 ms               | 5–15 ms                            |
| 4 | Kafka → stream processor | Consumer group, per-VIN ordering by key                       | 1–10 ms                            |
| 5 | Processing               | Dedup, 12 rules, trips, enrichment                            | ~8 μs/event                        |
| 6 | Alert → API / WebSocket  | Redis pub/sub (push) and alerts.v1 → sink-writer → PostgreSQL | 0.02–0.54 s to API; p95 480 ms     |
| 7 | Live state → dashboard   | Redis flush 500 ms; KPI snapshot + WebSocket push every 1 s   | vehicle ≤ ~0.8 s; KPI tiles ≤ ~2 s |
| 8 | Enriched → ClickHouse    | Kafka engine + materialised views                             | seconds (history only)             |
| 9 | Nightly                  | Features → model → risk scores → read model                   | daily batch                        |

## 5.2 Technology Stack & Justification

| Layer                                        | Choice                                                                                                                                                                                       | Why This, and What You Rejected                                                                                                                                                                                                                                                                  |
|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ingestion / Messaging                        | MQTT (Mosquitto / EMQX) at the edge, HTTPS push, <b>Apache Kafka</b> backbone                                                                                                                | Kafka: durable replayable log, per-VIN ordering, consumer groups, bursts absorbed, native ClickHouse engine. Rejected RabbitMQ (no replay, weaker ordering), cloud-native streams (lock-in).                                                                                                     |
| Stream / Batch Processing                    | Go stream processor (franz-go); Python batch jobs (pandas, ClickHouse SQL)                                                                                                                   | Go: ~120K events/s/core, bounded memory, tiny images. Rejected Flink/Kafka Streams (JVM ops weight for a small team) and Python on the hot path (10–20× cores).                                                                                                                                  |
| Relational / NoSQL / Cache / Search / Vector | PostgreSQL 16 + RLS + pgvector; ClickHouse; Redis                                                                                                                                            | Postgres for the 3NF core, transactions, RLS, vector search; ClickHouse for 8.6B rows/day (31 B/row); Redis for sub-ms live state. Rejected TimescaleDB (storage/write cost at scale), Cassandra (ad-hoc analytics), separate vector DB (pgvector sufficient at KB size).                        |
| Backend / Frontend                           | FastAPI (async, asyncpg, ORJSON); React + TypeScript + Tailwind + Recharts + Leaflet                                                                                                         | FastAPI: typed contracts, OpenAPI, async I/O, official Claude SDK. React: component ecosystem, charts, maps. Rejected server-rendered UI (live WebSocket dashboard).                                                                                                                             |
| ML / AI                                      | scikit-learn<br>HistGradientBoosting per component; provider-agnostic LLM tool use (any OpenAI-compatible endpoint, e.g. free Groq, Gemini, OpenRouter or local Ollama, or Claude); pgvector | Boosted trees handle missing sensor families and non-linearity, fast and explainable. The copilot works with free or self-hosted models and degrades to an offline planner, so no paid API is required. Rejected deep sequence models (data size, explainability) and lock-in to one LLM vendor. |

| Layer                                  | Choice                                                                                 | Why This, and What You Rejected                                                                                                                 |
|----------------------------------------|----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Infrastructure / CI-CD / Observability | Docker Compose, Helm, Terraform (AWS; GCP values), GitHub Actions, Prometheus, Grafana | One-command local stack; cloud-agnostic chart; CI with tests, scans, SBOMs, e2e. Rejected managed-only services in the data path (portability). |

## 5.3 Data Architecture

- **ER Diagram:** Relational core normalised to 3NF; list any deliberate denormalisation and why
- **Polyglot Map:** Which data lives in which store, with its CAP choice (CP or AP)
- **Capacity Estimate:** Events/sec, bytes/event, daily and yearly volume, partition keys, hot / warm / cold retention
- **Query Optimisation:** Top three queries with EXPLAIN ANALYZE before and after

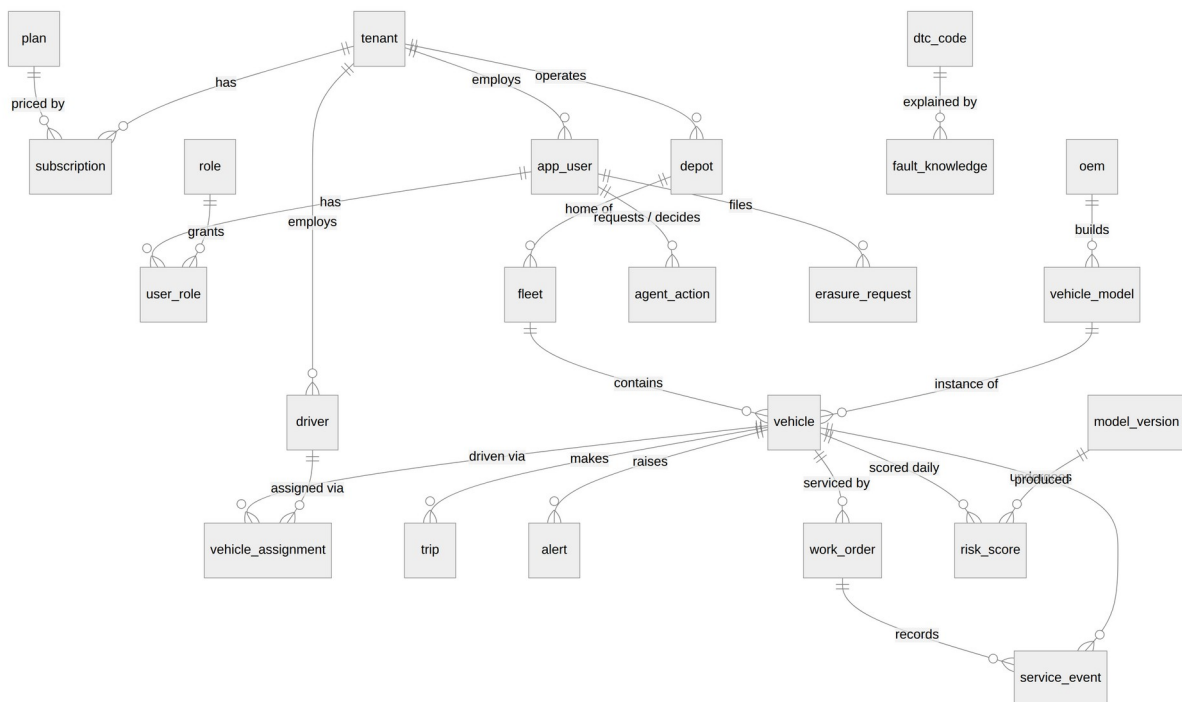


Figure 5. Relational core (PostgreSQL), 3NF; full attributes in docs/erd.md.

**Deliberate denormalisation:** tenant\_id is copied onto child tables (alerts, trips, work orders, risk scores) so row-level security filters each row without a join, guarded by composite foreign keys (fleet\_id, tenant\_id); trip is partitioned by month without an FK (high-volume append); rule evidence and model explanations are jsonb; three materialised read models (current risk, driver safety, daily alerts) form the CQRS read side.

## Polyglot map and CAP choices:

| Store         | Data                                                                                                   | CAP        | Why                                                                   |
|---------------|--------------------------------------------------------------------------------------------------------|------------|-----------------------------------------------------------------------|
| Kafka         | Raw, enriched, alerts, trips, DLQ                                                                      | CP         | RF 3, min.insync 2, acks=all: no silent loss; producers back-pressure |
| PostgreSQL 16 | Tenants, users, vehicles, drivers (encrypted PII), alerts, work orders, trips, risk, audit, KB vectors | CP         | Transactions, constraints, RLS; Multi-AZ failover                     |
| ClickHouse    | 1 Hz telemetry, features, roll-ups, rejects                                                            | AP-leaning | Async replication; ReplacingMergeTree converges duplicates            |
| Redis         | Live state, KPI snapshots, pub/sub, rate limits, cache                                                 | AP         | Rebuilt from the stream in ~15 s (chaos-tested)                       |

## Capacity estimate (100K vehicles at 1 Hz; sizes measured on the running stack):

| Item                  | Value                                                                                                                |
|-----------------------|----------------------------------------------------------------------------------------------------------------------|
| Events/s              | 100,000 sustained; 300,000 for 5 min at shift start                                                                  |
| Bytes/event           | 407 B JSON on the wire; 124 B in Kafka (lz4); 31.3 B in ClickHouse (4.9× compression)                                |
| Daily volume          | 8.64 B events; Kafka 1.07 TB/replica; ClickHouse 270 GB compressed                                                   |
| Yearly (steady state) | ClickHouse ≈ 24 TB (raw kept 90 d) + < 5 GB roll-ups; PostgreSQL ≈ 60 GB + audit log                                 |
| Partition keys        | Kafka: VIN (48 partitions); ClickHouse: day, ordered by (tenant_id, vin, ts, seq); trips: month                      |
| Hot / warm / cold     | Kafka 24 h replay; ClickHouse NVMe 7 d → S3 cold to 90 d → delete; roll-ups 2 y; PostgreSQL trips 13 months (policy) |

## Query optimisation (EXPLAIN ANALYZE as the least-privilege API role with RLS active; plans in docs/evidence/explain/):

| Query                                                          | Before (ms) | After (ms) | Change Made                                                                                                                   |
|----------------------------------------------------------------|-------------|------------|-------------------------------------------------------------------------------------------------------------------------------|
| Vehicle list page (50 rows + current risk), 55K-vehicle tenant | 351.1       | 1.3        | Security-barrier view blocked join push-down (scan + sort of 55K rows); fenced LATERAL (OFFSET 0) probe uses the unique index |
| Riskiest vehicles, top 50                                      | 2,910.7     | 7.3        | Materialised current-risk read model instead of DISTINCT ON over 1.1M scores                                                  |
| Driver safety ranking, 30 days                                 | 2,009.7     | 36.5       | mv_driver_safety read model, refreshed hourly                                                                                 |
| Alert trend, 14 days                                           | 1,001.1     | 0.5        | mv_alert_daily for completed days +                                                                                           |

| Query                             | Before (ms)    | After (ms)  | Change Made                                                                         |
|-----------------------------------|----------------|-------------|-------------------------------------------------------------------------------------|
|                                   |                |             | live count for today                                                                |
| Vehicle list, page 1,000          | 40.5           | 1.2         | Keyset pagination (HMAC-signed cursor) instead of OFFSET 50000                      |
| ClickHouse: one vehicle, last 6 h | 8,386 granules | 13 granules | Filter on sort-key prefix (tenant_id, vin, ts); never wrap key columns in functions |

## 5.4 Deployment View

- **Deployment Diagram:** Clusters, namespaces, replicas, autoscaling, networking, secrets
- **Cloud-Agnostic Approach:** How the same deployment runs on a second cloud or locally

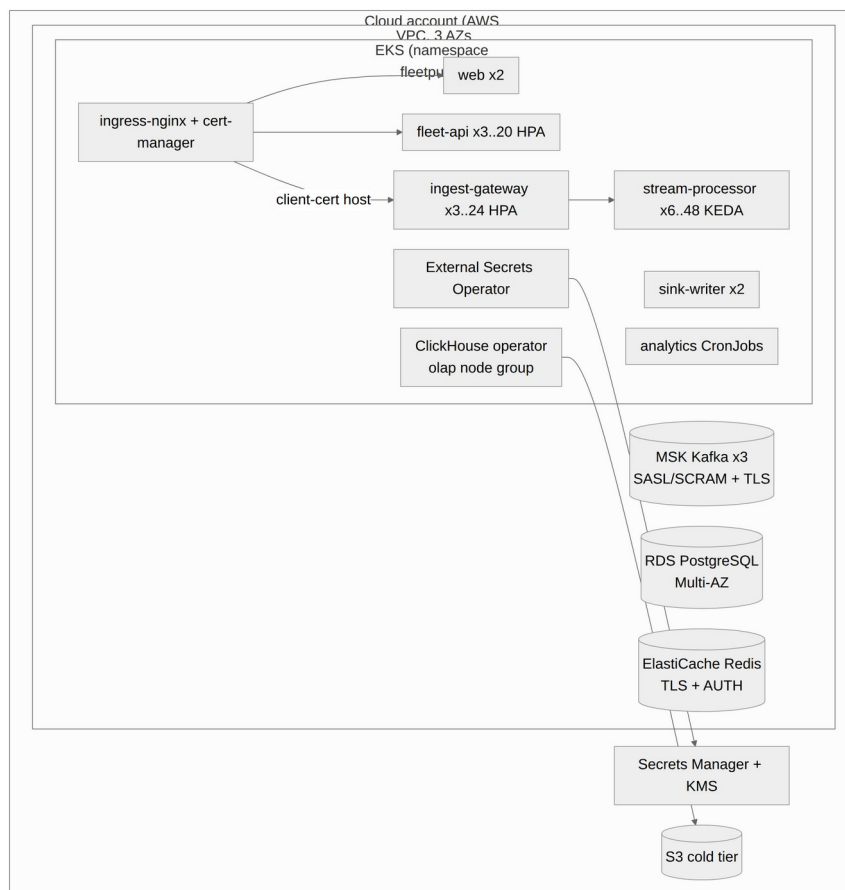


Figure 6. Deployment on AWS (Terraform + Helm).

- **Cluster and namespaces:** EKS across 3 AZs, one namespace per environment; node groups for stream, apps and ClickHouse (tainted); managed MSK, RDS PostgreSQL Multi-AZ, ElastiCache Redis, S3 cold tier.

- **Replicas and autoscaling:** ingest-gateway 3-24 (HPA, CPU), stream-processor 6-48 (KEDA on consumer lag, capped by partitions), fleet-api 3-20 (HPA), sink-writer 2, web 2; PDBs, zone and node spreading, zero-downtime rolling updates; analytics as CronJobs; migrations as a Helm hook before every upgrade.
- **Networking:** default-deny NetworkPolicies with explicit allows; ingress-nginx with cert-manager TLS; a separate ingest host with client-certificate verification; instance metadata blocked.
- **Secrets:** generated by Terraform into Secrets Manager (KMS), synced by External Secrets Operator via IRSA; pods run non-root with read-only root filesystems.
- **Cloud-agnostic:** the chart needs only endpoints and a secret store: values-gcp.yaml targets Cloud SQL, Memorystore, Managed Kafka and GCS; docker compose up runs the same images locally. Every component in the data path is open source, so managed services are substitutes, not dependencies.

## 6. Low-Level Design

### 6.1 Layering & Separation of Concerns

- **Architecture Style:** Layered, hexagonal (ports and adapters) or clean architecture — justify
- **Layer Diagram:** Internal layers of at least one core service and the allowed dependency direction
- **Folder Structure:** Repository tree, two levels deep

**Architecture style: hexagonal (ports and adapters).** Detection rules, dedup, trip segmentation, VIN/DTC parsing and the ML model are pure domain code with no I/O, tested without infrastructure; Kafka, Redis, PostgreSQL and ClickHouse sit behind narrow ports (Output, Live, Meta, Sink). This let us unit-test 84 pipeline cases in milliseconds and swap transports (MQTT/HTTPS) without touching rules.

| Layer                 | Responsibility                                                                                                           | Must Not                                |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| Presentation / API    | HTTP, WebSocket, validation, auth checks, DTO mapping<br>FleetPulse: fleet-api/app/api/*, cmd/*                          | Contain business rules or SQL           |
| Application / Service | Use cases, transactions, orchestration<br>FleetPulse: app/services/*, internal/processor, analytics/jobs.py              | Depend on a specific database or broker |
| Domain                | Entities, value objects, business rules<br>FleetPulse: internal/detect, stream, vin, dtc; analytics/model.py             | Import framework or infrastructure code |
| Infrastructure        | Repositories, broker clients, external APIs, caches<br>FleetPulse: internal/platform, processor/adapters.go, app/infra/* | Leak vendor types into the domain       |

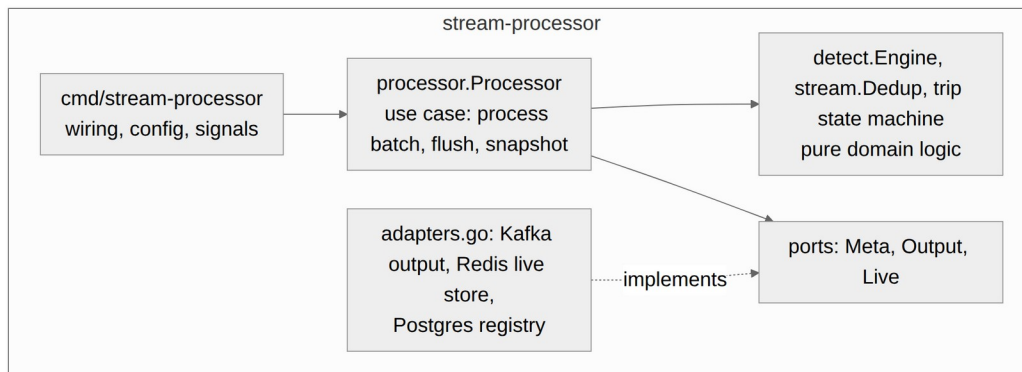


Figure 7. Stream-processor layers; dependencies point inwards.

## Folder structure (two levels):

|                     |                                                                 |
|---------------------|-----------------------------------------------------------------|
| contracts/          | JSON Schemas per topic, oem-samples/                            |
| db/                 | postgres/migrations, clickhouse                                 |
| deploy/             | helm/fleetpulse, mosquitto, clickhouse, observability           |
| docs/               | architecture, adr/, diagrams/, evidence/, screenshots/, api/    |
| infra/terraform/    | aws/                                                            |
| scripts/            | explain-evidence.sh, send-sample.py                             |
| services/pipeline/  | cmd/ (simulator, ingest-gateway, stream-processor, sink-writer, |
| migrate), internal/ |                                                                 |
| services/fleet-api/ | app/ (api, services, security, infra, agent, domain), tests/    |
| services/analytics/ | analytics/, tests/                                              |
| services/web/       | src/ (pages, components, lib, api)                              |
| tests/              | bdd/, load/, chaos/                                             |

## 6.2 Design Principles Applied

- **SOLID:** single responsibility per package (vin, dtc, detect, stream); open/closed OEM registry (new adapters by spec, no code change); Liskov and interface segregation via small ports (Output, Live, Sink, Publisher) with fakes in tests; dependency inversion: the processor depends on ports, cmd/\* wires concrete adapters. Result: fast unit tests and transport swaps without touching rules.
- **12-Factor App:** all configuration from the environment (config.py, platform.Env\*, .env.example); stateless processes (state in Kafka, Redis, databases); disposability with graceful shutdown (offsets committed on revoke, audit queue drained); logs to stdout; dev/prod parity through the same images in Compose and Helm; admin processes as one-off jobs (migrate, seed).
- **Idempotency:** deterministic event, alert and trip IDs with ON CONFLICT, ReplacingMergeTree, one pending AI proposal per vehicle. **Fail-fast:** schema/VIN validation at the edge, circuit breakers, startup checks. **Least privilege:** separate DB roles (API without BYPASSRLS, writer, migrator),

RBAC permissions per endpoint and per copilot tool, per-service IAM roles.

**DRY:** one canonical event, shared JSON Schemas, one embedding function used by API and batch (contract-tested). **KISS:** JSON + schemas instead of a registry, hashing embeddings instead of an embedding service.

### 6.3 Design Patterns Used

| Pattern                       | Problem It Solves in Your System                                                                        | Location in Code                                           |
|-------------------------------|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| Adapter                       | Maps each OEM's payload format (nested/flat/signal list, units, time) to one canonical event            | internal/oem/ (specs/*.json, mapping.go, stellar.go)       |
| Circuit Breaker / Retry       | Protects the API when ClickHouse is slow or down (fast degraded answers); producers retry with back-off | fleet-api/app/infra/ breaker.py; sim/runner.go             |
| CQRS (read models)            | Heavy aggregates served from materialised views refreshed by jobs                                       | db/postgres/ migrations/ 005_read_models.sql               |
| Ports & adapters / Repository | Domain logic independent of Kafka/Redis/PostgreSQL                                                      | internal/processor (Output, Live, Meta), app/services/*    |
| Idempotent consumer           | At-least-once delivery without duplicate effects                                                        | detect.alertID, sink ON CONFLICT, ReplacingMergeTree       |
| Observer / Pub-Sub            | Alerts pushed to every connected manager of the tenant                                                  | processor → Redis alerts:{tenant} → api/fleet.py WebSocket |
| Strategy / Registry           | Per-OEM decoders and per-rule detectors selected at runtime                                             | oem.Registry, detect.Engine                                |
| State machine                 | Trip segmentation and vehicle status                                                                    | internal/detect/ detect.go                                 |
| Bulkhead / back-pressure      | Bounded in-flight batches, 429 + Retry-After instead of overload                                        | ingest/gateway.go (acquire)                                |
| Cache-aside                   | Tenant-scoped response cache with TTL                                                                   | fleet-api/app/infra/ cache.py                              |
| Human-in-the-loop command     | AI proposes, a manager approves, then it executes                                                       | agent/tools.py, api/copilot.py                             |

### 6.4 Interfaces, Contracts & Runtime Flows

- **API Contract:** OpenAPI / AsyncAPI link; versioning, pagination, error format, rate limits
- **Event Schemas:** Topics, keys, schema format (Avro / Protobuf / JSON Schema), schema evolution
- **Sequence Diagrams:** Two critical flows, including one failure path (broker down, duplicate event, OEM format change)

- **API contract:** OpenAPI 3 at /docs and docs/api/openapi.json (33 paths). Versioned under /api/v1; opaque HMAC-signed keyset cursors (next\_cursor); errors as RFC 9457 application/problem+json with a request\_id; per-user sliding-window rate limit (1,200/min) with X-RateLimit-Remaining and Retry-After; optimistic concurrency with If-Match on work orders; WebSocket stream with one-time tickets.
- **Event schemas:** JSON with JSON Schema per topic in /contracts; topic names carry the version.

| Topic                 | Key | Producer → consumers                       | Retention (prod) |
|-----------------------|-----|--------------------------------------------|------------------|
| telemetry.v1          | VIN | ingest-gateway → stream-processor          | 24 h (replay)    |
| telemetry.enriched.v1 | VIN | stream-processor → ClickHouse              | 6 h              |
| alerts.v1             | VIN | stream-processor → sink-writer, ClickHouse | 7 d              |
| trips.v1              | VIN | stream-processor → sink-writer             | 7 d              |
| telemetry.dlq.v1      | OEM | ingest-gateway → ClickHouse ingest_rejects | 7 d              |

**Schema evolution:** additive fields are backward compatible; breaking changes get a new .v2 topic with dual-write during migration. Contract tests fail CI if a producer struct or the ClickHouse consumer DDL drifts from a schema.

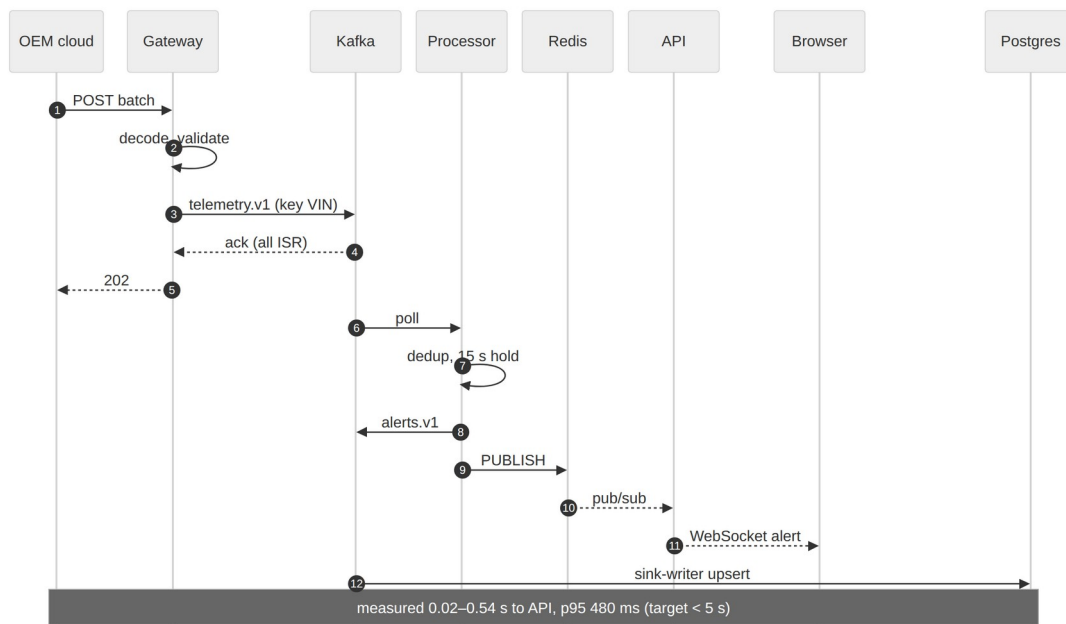


Figure 8. Critical flow: overheating vehicle to the manager's screen.

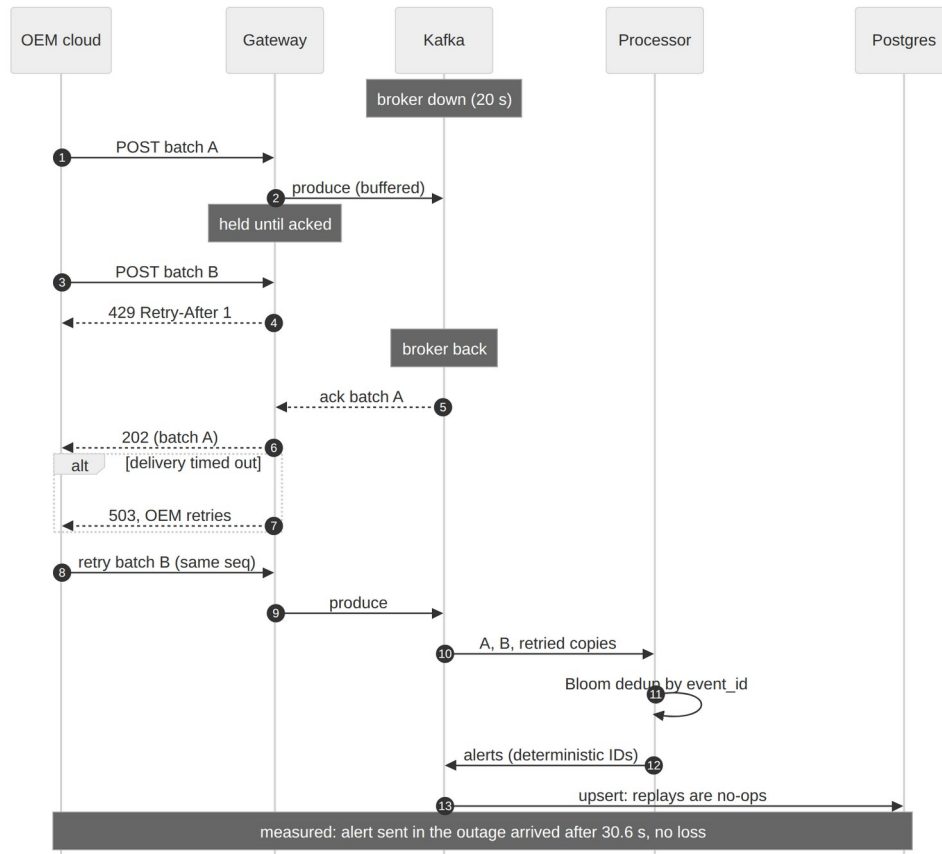


Figure 9. Failure path: Kafka unavailable, OEM retries, duplicates absorbed.

In the failure path the gateway never acknowledges before Kafka has the data; above the producer high-water mark it answers 429 with Retry-After, on delivery timeout 503. Retried records keep their (OEM, VIN, sequence) and hence their event\_id, so the Bloom filter drops them and alert IDs stay deterministic. OEM format changes follow a third path: records that no longer match the spec are rejected individually to the DLQ with the reason, and the spec is updated without a code release.

## 6.5 Algorithms & Data Structures

- **Problem Solved:** e.g. nearest reachable charger, trip segmentation, VIN / DTC parsing
- **Algorithm Used:** Graph, dynamic programming, regex, streaming structures — justify the choice
- **Pseudocode:** For major functions
- **Time & Space Complexity:** Annotate all
- **Scale Tested:** Input size and measured runtime

| Problem               | Algorithm / structure                     | Time  | Space | Measured |
|-----------------------|-------------------------------------------|-------|-------|----------|
| Corrupted vehicle IDs | VIN check digit (ISO 3779 weights mod 11) | O(17) | O(1)  | 319 ns   |

| Problem               | Algorithm / structure                                | Time                                  | Space                                    | Measured                            |
|-----------------------|------------------------------------------------------|---------------------------------------|------------------------------------------|-------------------------------------|
| Fault-code spellings  | Regex normalisation to [PCBU] [0-3][0-9A-F]{3}       | $O(\text{len})$                       | $O(1)$                                   | in decode                           |
| OEM retries / replays | Rotating pair of Bloom filters (Kirsch-Mitzenmacher) | $O(k)$                                | $O(n)$ bits, bounded                     | 275-424 ns                          |
| Trending fault codes  | Count-Min Sketch + min-heap top-K                    | $O(d + \log K)$                       | $O(w \cdot d + K)$                       | —                                   |
| Density map           | Geohash precision 3-5                                | $O(p)$                                | $O(1)$                                   | 123-148 ns                          |
| Sustained faults      | Event-time hold timer + cooldown per rule            | $O(1)/\text{event}$                   | $O(\text{vehicles} \times \text{rules})$ | ~8 $\mu\text{s}/\text{event}$ total |
| Trips, idling         | State machine PARKED/IDLING/DRIVING                  | $O(1)$                                | $O(\text{vehicles})$                     | (same)                              |
| Deep paging           | Keyset pagination, signed cursor                     | $O(\log n + \text{page})$             | $O(1)$                                   | 1.2 ms at page 1,000                |
| 7-day risk            | Gradient-boosted trees per component + noisy-OR      | $O(\text{trees} \times \text{depth})$ | $O(\text{model})$                        | 100K vehicles/job                   |
| Knowledge search      | Signed feature hashing (256-d) + HNSW                | $O(\text{tokens}); \sim O(\log n)$    | $O(n \cdot d)$                           | —                                   |

## Pseudocode (major functions):

```

Dedup.Seen(key):
    # two generations: current, previous
    if now - rotatedAt > period: previous = current; current = empty; rotatedAt = now
    if current.Test(key) or previous.Test(key): return true    # 0(k), no false negatives
    current.Add(key); return false
    # n = 2M keys/partition per 2 min, p = 0.001 -> m = 28.8M bits (3.6 MB), k = 10

Rule.Observe(e, v):
    # e.g. coolant >= 110 C held 15 s
    if cond(e):
        v.since = v.since or e.ts    # event time, not arrival time
        if e.ts - v.since >= hold and e.ts - v.lastFired >= cooldown:
            emit Alert(id = uuid5(vin|rule|e.ts/cooldown))    # replays -> same id
            v.lastFired = e.ts
        else: v.since = None    # late events never touch rules or live state

Risk(features):
    for c in {cooling, battery12v, misfire, ev_pack}:
        p[c] = GBT_c(features)
        p[c] = p[c]*r / (p[c]*r + 1 - p[c])    # prior correction, r = 0.25
        if impossible(c, powertrain): p[c] = 0
    return 1 - prod(1 - p[c]), argmax p, top feature reasons
  
```

**Scale tested:** 100,000 vehicles with 60 days of history (74M telemetry rows, 2.0M trips, 536K alerts, 1.1M risk scores) seeded in ~85 s; single-core benchmarks: processor 116-134K events/s, gateway ~100K records/s; 30,025 API requests at 100 req/s; ~4K events/s end to end on 4 vCPU for over an hour with bursts, duplicates, out-of-order data and outages.

## 7. Non-Functional Requirements & Performance Benchmarks

| NFR                | Target (Case Study)                   | Achieved                                                                                               | How Measured                                                                                                   |
|--------------------|---------------------------------------|--------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Ingest Throughput  | 100K+ events/sec                      | Per core:<br>~120K (processor),<br>~100K (gateway).<br>End to end:<br>~4K ev/s on a shared 4-vCPU host | Go benchmarks + Prometheus; 100K sized as 3 gateway + 6-48 processor pods. Not demonstrated end to end at 100K |
| End-to-End Latency | < 2 s dashboard; < 5 s critical alert | Alert p95 480 ms (0.02-0.54 s);<br>vehicle ≤ 0.8 s; KPIs ≤ ~2 s                                        | BDD + chaos sentinel alerts;<br>processor_alert_latency_seconds; flush/push intervals                          |
| API Latency        | p95 < 200 ms; p99 < 500 ms            | p95 135 ms;<br>p99 408 ms;<br>0 errors                                                                 | k6, 100 req/s × 5 min, 12 users, 3 tenants, 9 endpoints                                                        |
| Resilience         | Recovers after broker / pod failure   | 5/5 chaos pass; no data loss                                                                           | chaos.py: processor kill → alert in 10.9 s; 20 s Kafka outage → no loss; Redis wipe → rebuilt in 15 s          |
| Availability       | 99.9%, no single point of failure     | Designed, not measured                                                                                 | 3 AZs, Kafka RF 3, Multi-AZ RDS/Redis, ≥ 2 replicas + PDBs, HPA/KEDA; no long-running deployment to measure    |

- **Load Test Setup:** Tool, data volume, hardware / cluster size
- **Results:** Graphs or screenshots of throughput, p95 / p99 and consumer lag

**Load test setup:** k6 0.55 constant-arrival-rate against the API (dashboard mix: vehicle detail and lists, alerts, fleet summary, risk ranking, live clusters, top fault codes, analytics), 12 demo users across 3 tenants, seeded dataset of 100K vehicles; everything (Kafka, ClickHouse, PostgreSQL, Redis, pipeline, simulator, API with 4 workers, Grafana, k6) on one 4-vCPU / 16 GB host. Pipeline: simulator at ~4K events/s with 3× bursts, 1% duplicates, 2% out-of-order and periodic outages.

| Endpoint (100 req/s, 5 min) | Requests | p50 ms | p95 ms | p99 ms |
|-----------------------------|----------|--------|--------|--------|
| GET /vehicles/{vin}         | 7,449    | 15.3   | 146.9  | 480.5  |
| GET /vehicles               | 6,046    | 28.8   | 159.3  | 429.6  |
| GET /alerts                 | 4,527    | 16.8   | 119.2  | 353.0  |
| GET /fleet/summary          | 3,084    | 8.3    | 97.6   | 371.2  |
| GET /maintenance/risk       | 2,917    | 37.0   | 185.7  | 396.7  |

| Endpoint (100 req/s, 5 min)                                                     | Requests | p50 ms  | p95 ms | p99 ms  |
|---------------------------------------------------------------------------------|----------|---------|--------|---------|
| GET /live/clusters, /live/top-dtc, /maintenance/summary, /analytics/alert-trend | 5,978    | 7.4-8.1 | 58-95  | 238-424 |

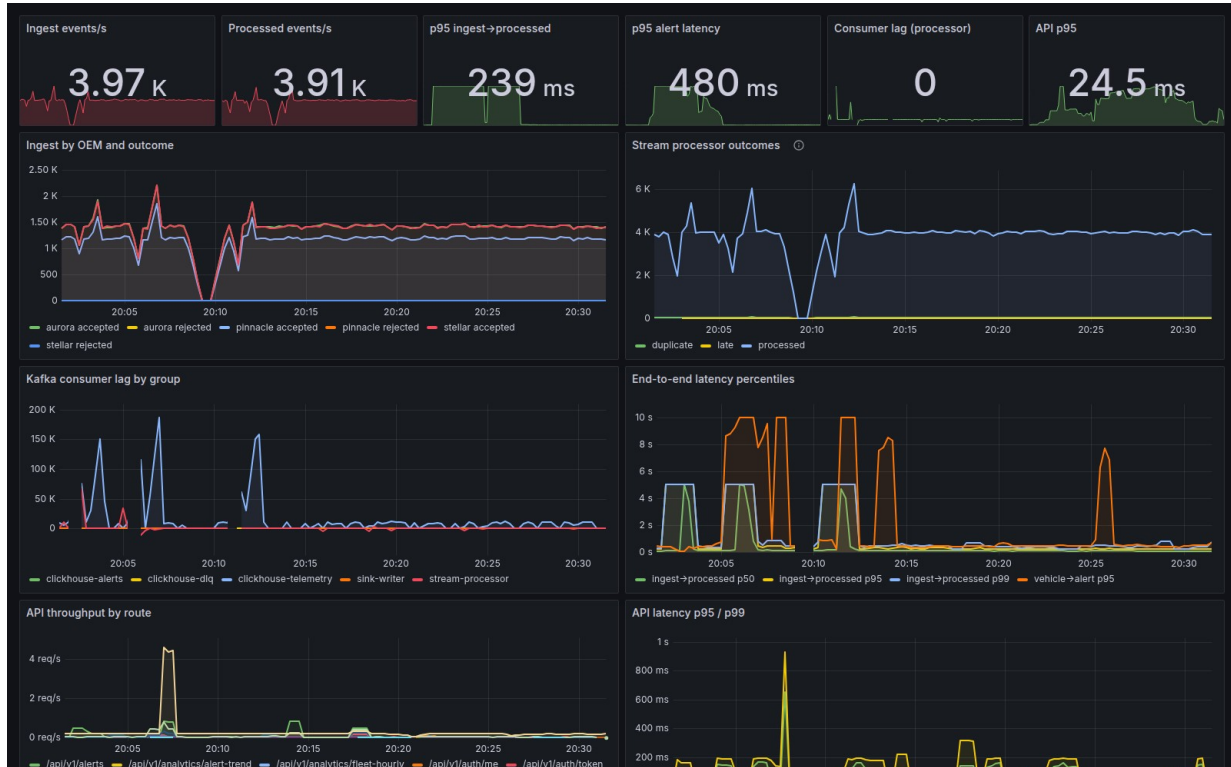


Figure 10. Grafana during the test and chaos runs: ingest vs processed events/s, p95 ingest→processed 239 ms, p95 alert latency 480 ms, consumer lag, API latency (dips are the injected faults).

The API is stateless and CPU-bound (~11 ms CPU per request on this host); throughput scales linearly with replicas. At 150 req/s on the same host p95 exceeded the target because all components shared four cores; that run exposed and led to fixing the 351 ms vehicle-list query.

## 8. Security & Compliance

- **Threat Model:** STRIDE for the ingestion path and public API; top five threats and controls
- **Authentication & Authorisation:** OAuth2 / OIDC, RBAC, tenant isolation
- **Device Identity & Encryption:** mTLS, TLS in transit, encryption at rest, secrets management
- **Privacy:** How location and driver data is minimised, masked, retained and erased (GDPR, DPDP Act 2023)
- **AI Safety (if applicable):** Prompt-injection defences, tool permissions, human approval, audit log

## Threat model (STRIDE) for the ingestion path and public API: top threats and controls

| Threat                                 | Where     | Control                                                                                                                                       | Evidence                                               |
|----------------------------------------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| Spoofing a vehicle / OEM connector     | Ingestion | mTLS client certs (gateway MQTT_TLS_*), per-account broker ACL; OEM API key (constant-time) + client-cert ingress; VIN check digit + registry | Gateway auth tests; BDD 'Connectors must authenticate' |
| Cross-tenant disclosure                | API       | Tenant claim in RS256 JWT → app.tenant_id per transaction → RLS (API role without BYPASSRLS); ClickHouse row policy; 404 for foreign IDs      | test_tenant_isolation_by_rls; BDD tenant scenario      |
| Tampering / repudiation of records     | API, DB   | Every request and AI tool call audited; hash-chained, append-only audit table; verify endpoint                                                | test_audit_tamper_detection                            |
| Denial of service                      | Both      | Bounded in-flight batches → 429/503; Kafka buffering; per-user rate limit; login throttle + lockout                                           | k6 ratelimit.js; TestHTTP_BackPressure                 |
| Elevation of privilege via the copilot | API       | Permission-filtered tools executing under RLS; proposals need approval; injection screen; grounding check; tool budget                        | Guardrail + approval tests; BDD                        |
| Personal-data disclosure               | API       | Driver pseudonyms without driver:pii; 5 km location grid without location:precise; AES-256 PII columns                                        | test_rbac_and_location_masking; BDD                    |

**Authentication & authorisation.** OAuth2 password grant issuing RS256 JWTs (60 min, issuer/audience checked, revocable by token ID) with an OpenID configuration and JWKS endpoint; five roles mapped to fourteen permissions enforced per endpoint; tenant isolation enforced in the database, so a query bug cannot leak across tenants. External OIDC federation is a planned extension.

**Device identity & encryption.** mTLS for devices/OEM connectors in production (local stack uses broker passwords and ACLs); TLS 1.2+ to Kafka (SASL/SCRAM), PostgreSQL (verify-full), Redis and ClickHouse; KMS AES-256 at rest for every managed store, volume and bucket; secrets generated into Secrets Manager and synced by External Secrets Operator (never in Git or images; gitleaks in CI).

**Privacy (GDPR, DPDP Act 2023).** Telemetry keyed by VIN, not person; precise location only for roles that need it; driver names only with driver:pii; raw telemetry deleted after 90 days; right to erasure overwrites identity, destroys encrypted licence and phone, unlinks and coarsens trips, purges caches and returns a per-store report (idempotent, audited).

**AI safety.** Prompt-injection screen before the model; tools restricted to the user's permissions and RLS; write actions only as proposals approved by a maintenance manager (viewers get 403); ungrounded VINs redacted; tool-call and token

budgets; every question, tool call and proposal in the audit chain; any provider error or rate limit falls back to the offline planner behind a circuit breaker.

**Security testing in CI:** gitleaks, Semgrep (OWASP Top 10), govulncheck, npm audit, Trivy (filesystem, IaC, images; fail on fixable criticals), CycloneDX SBOMs, OWASP ZAP baseline with header/cookie/CORS rules set to fail. These scans are configured in the pipeline but were not executed on the build machine.

## 9. Test Strategy

| Test Type                                | Tools                                                                             | No. of Tests                            | Coverage / Result                                                        | In CI?                       |
|------------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------|--------------------------------------------------------------------------|------------------------------|
| Unit                                     | go test -race, pytest, fakeredis, httpx mock, Vitest                              | 84 Go, 63 API, 10 analytics, 17 web     | Coverage: Go 84.3%, API 88%, analytics 90% (gates 80%)                   | Yes                          |
| Integration & Contract                   | Testcontainers (PostgreSQL + pgvector, Redis); JSON Schema contract tests         | 17 API, 6 analytics, 2 contract suites  | All pass; contracts validate 3,000 events, 689 alerts, 321 trips per run | Yes                          |
| Acceptance (BDD)                         | behave (Gherkin) against the running stack                                        | 3 features, 9 scenarios, 32 steps       | 9/9 pass; alert visible 0.15-0.33 s                                      | Yes (e2e job)                |
| Performance / Load / Soak                | k6 (load, smoke, rate limit, soak profile); Go benchmarks                         | 4 scripts/pro files, 6 benchmarks       | 100 req/s: p95 135 ms, p99 408 ms, 0 errors; soak not run                | Smoke                        |
| Security (SAST, DAST, Dependency, Image) | Semgrep, ZAP baseline, govulncheck, npm audit, Trivy, gitleaks                    | 6 scanners                              | Configured in CI; not run on the build machine                           | Yes                          |
| Compliance & Chaos                       | Integration tests (RLS, masking, audit tamper, erasure); chaos.py (Docker faults) | 6 compliance tests, 5 chaos experiments | All pass                                                                 | Compliance yes; chaos manual |

- **Edge Cases Covered:** e.g. duplicate events, out-of-order data, unknown OEM format, broker outage
- **Sample Input / Output:** If helpful
- **Report Links:** Coverage, load-test and security scan reports

**Edge cases covered:** duplicate events across transports; out-of-order and late data; malformed JSON, bad VIN check digits, impossible readings, clock skew; unknown OEM and unknown vehicle; a new OEM format without code; unit conversions (mph, °F, psi, K, m/s); gzip payloads; Kafka down and back-pressure; processor crash mid-stream; ClickHouse down; Redis wiped; cross-tenant access; JWT forgery (alg none, foreign key, wrong audience, expiry); cursor tampering; concurrent work-order edits; audit tampering; repeated erasure; prompt injection and ungrounded VINs; label leakage in ML; EVs cannot fail on cooling.

**Sample input / output:** scripts/send-sample.py posts one payload per OEM format:

```
$ scripts/send-sample.py
aurora    202 {"accepted":2,"rejected":0}
pinnacle 202 {"accepted":2,"rejected":0}
stellar   202 {"accepted":2,"rejected":0}
$ scripts/send-sample.py pinnacle --corrupt
pinnacle 202 {"accepted":1,"rejected":1} # DLQ: vin "...": vin: check digit mismatch
```

**Report links** (repository): docs/testing.md, docs/evidence/ (k6 summaries and per-endpoint latency, chaos report, EXPLAIN plans, pipeline benchmarks); CI artifacts: coverage XML, JUnit, SBOMs, BDD reports.

## 10. Observability

- **Dashboard Screenshot:** Throughput, consumer lag, error rate, latency percentiles
- **Troubleshooting Walk-Through:** How you would find the cause of a latency spike using logs, metrics and traces

**Signals:** Prometheus metrics from every service (ingest events by OEM/outcome, back-pressure, processor outcomes, ingest→processed and alert latency histograms, partitions owned, sink persist time, Kafka consumer lag, API latency by route, circuit-breaker state, copilot tokens and latency, analytics job success and model metrics); eight alert rules (lag, latencies, error rate, reject spike, breaker open, back-pressure); structured JSON logs with request\_id returned in X-Request-ID and every error; optional OpenTelemetry tracing for the API; events carry event\_id, ingest\_ms and proc\_ms end to end. The dashboard is shown in Figure 10 (section 7).

### Troubleshooting walk-through: a latency spike.

1. API or pipeline? Grafana API p95/p99 by route: one route spiking means a query; all routes means saturation or a dependency.
2. Dependency? Circuit-breaker gauges and /api/v1/admin/dependencies show which store is slow or open.

**3. Query?** Take the request\_id from the slow response, find its log line (route, tenant, duration), check pg\_stat\_statements, then EXPLAIN ANALYZE as the API role with the tenant set. This is how the 351 ms security-barrier join was found during load testing.

**4. Pipeline?** Consumer lag by group, processor outcomes (a jump in unknown\_vehicle or late is data, not capacity), ingest by OEM and outcome (format change → DLQ reasons). This is how we found MQTT messages lost to orphaned shared-subscription sessions: gateway intake was one third of the simulator's rate.

**5. Capacity?** CPU vs HPA targets, back-pressure counter, lag on all partitions → scale out (KEDA does it); lag on one partition → hot key or stuck consumer.

## 11. AI / ML Component (if used)

- **Purpose:** The decision the model or agent supports, and why rules alone were not enough
- **Data & Features:** Training or retrieval data, how it was generated or sourced, leakage checks
- **Model / Agent Design:** Model choice, vector store, tools the agent can call, how access is limited
- **Evaluation:** Metrics against a simple baseline
- **Guardrails & Cost:** Failure handling, hallucination checks, human-in-the-loop, cost and latency per request

**Purpose.** The model answers "which vehicles should go to the workshop this week?" at a fixed workshop capacity. Rules fire on thousands of healthy vehicles and miss slow trends (a coolant average creeping 9 °C over two weeks, a charging voltage sagging below the vehicle's own baseline). The copilot turns cross-screen questions into tool calls and queues approved actions.

**Data & features.** 60 days of synthetic telemetry for 100K vehicles with hidden, stochastically degrading component health (cooling, 12 V battery/charging, misfire, EV pack), noisy sensors, unrelated DTCs; breakdowns recorded as service events. 27 features per vehicle-day from ClickHouse (3/7/14-day windows, trends vs own baseline, DTC counts by system, usage, age, powertrain). Label: breakdown in (d, d+7]. Leakage checks: time-based split (train 15 Aug–7 Sep, test 8–22 Sep), features only up to day d, last 8 days unlabelled, unit test pinning the label window.

**Model / agent design.** One HistGradientBoostingClassifier per component, noisy-OR combination, negatives down-sampled to 25% and prior-corrected, impossible components zeroed, top feature contributions rendered as reasons. Copilot: a provider-agnostic manual tool-use loop over any OpenAI-compatible chat-completions endpoint (free tiers of Groq, Google Gemini or OpenRouter, a local Ollama) or Claude's Messages API, selected by configuration; 8 tools (fleet

summary, at-risk vehicles, vehicle health, alerts, fault knowledge search, idle cost, driver safety, propose work order), each mapped to a permission and executed under RLS; knowledge base in pgvector (HNSW, cosine) with deterministic 256-d feature-hashing embeddings.

**Evaluation** (held-out later period, 638,358 vehicle-days, base rate 2.0%, budget  $k = 1\%$  of vehicle-days):

| Method                   | PR-AUC | ROC-AUC | Precision @k | Recall@k | Net savings |
|--------------------------|--------|---------|--------------|----------|-------------|
| FleetPulse model         | 0.660  | 0.871   | 0.908        | 0.453    | \$8.28M     |
| Threshold alert rules    | 0.211  | 0.777   | 0.411        | 0.205    | \$4.31M     |
| Any fault code in 7 days | 0.183  | 0.755   | 0.429        | 0.214    | \$4.07M     |

Per component PR-AUC: cooling 0.70, 12 V battery 0.70, EV pack 0.58, misfire 0.46. Top-decile calibration: 14.8% predicted vs 15.0% observed. The data is synthetic: the comparison with baselines is the meaningful result, and pilot validation is the first next step.

**Guardrails & cost.** Failure handling: model fallbacks, then a deterministic offline planner using the same tools. Hallucination checks: VINs must appear in the question or tool results, otherwise redacted. Human in the loop: proposals approved by a maintenance manager; one pending proposal per vehicle. Limits: 2,000-character input, 6 tool calls per question, bounded output tokens. Cost: token usage per answer recorded and priced (free endpoints default to \$0), exported to Prometheus. Latency: offline answers 10–100 ms (measured). The LLM loop is unit-tested against a mocked OpenAI-compatible server and was run end to end through the API against a stand-in server with the real tools; no live provider was called on the build machine (no key). It is off every real-time path.

## 12. Architecture Decisions, Risks & Future Enhancements

- **Architecture Decision Records:** 3–5 records: Context → Options Considered → Decision → Consequences (cover databases, messaging and one CAP trade-off)
- **Risks & Technical Debt:** Known limitations and shortcuts taken during the hackathon
- **Future Enhancements:** Next three steps from prototype to pilot

**Architecture Decision Records** (full text in docs/adr/):

| ADR                            | Context                                      | Options considered                               | Decision                                                                | Consequences                                                                    |
|--------------------------------|----------------------------------------------|--------------------------------------------------|-------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| 0001 Event backbone            | 100K ev/s, bursts, several consumers, replay | Kafka, RabbitMQ, MQTT only, cloud streams        | Kafka after the edge; MQTT at the edge                                  | Per-VIN ordering, 24 h replay; partitions cap parallelism (48)                  |
| 0002 Telemetry store           | 8.6B rows/day plus a relational core         | ClickHouse, TimescaleDB, Cassandra, all-Postgres | ClickHouse for telemetry, PostgreSQL for the core, Redis for live state | 31 B/row, 13 of 8,386 granules per vehicle query; two isolation mechanisms      |
| 0003 CAP per store (trade-off) | Partitions force consistency vs availability | Uniform CP or AP                                 | CP: Kafka, PostgreSQL; AP: Redis, ClickHouse                            | Ingest back-pressures in a quorum loss; dashboards stay up with brief staleness |
| 0004 Languages                 | Hot path vs ML/API velocity                  | Go+Python, all Python, all Go, JVM               | Go pipeline, Python API/ML, React UI                                    | ~120K ev/s/core; API scales horizontally; explicit contracts                    |
| 0005 Contracts                 | Cross-language topics; OEM formats change    | Avro/Protobuf + registry, JSON + JSON Schema     | JSON Schema in repo, contract tests, versioned topics                   | Breaking changes fail CI; registry can come later                               |
| 0006 Multi-tenancy             | Competing fleets, leak = highest impact      | DB per tenant, schema per tenant, shared + RLS   | Shared schema + RLS, security-barrier views, ClickHouse row policy      | Bugs cannot leak data; barrier views need fenced LATERAL (351→1.3 ms)           |

## Risks & technical debt.

- 100K events/s end to end not demonstrated (4-vCPU build host); sized from per-core benchmarks. Next: a cloud load run.
- Model trained on synthetic data; needs validation on a pilot fleet's history.
- audit\_log not partitioned; PostgreSQL retention (old trip partitions, risk history) is policy, not yet automated.
- External OIDC federation, per-tenant quotas and driver safety opt-out enforcement are planned.
- Local MQTT uses passwords (mTLS supported and configured for production); single Kafka broker locally.
- Security scans and the 2-hour soak are configured but were not executed on the build machine; the copilot's LLM loop was tested against mocked and stand-in providers, not a live one.

## Future enhancements (prototype → pilot).

**1.** Pilot with one fleet: connect real OEM feeds through the adapter specs, back-test the model on their service history, calibrate costs, and run the 100K events/s load test on the Helm/Terraform deployment.

**2. Harden operations:** audit-log partitioning with WORM archive, automated retention jobs, per-tenant quotas and SSO (OIDC federation), SLO-based alerting and on-call runbooks.

**3. Extend the decision layer:** parts and workshop-slot optimisation, warranty-claim evidence for OEMs, EV charging and range risk, and closed-loop learning from work-order outcomes.

## 13. Demo Video (5 Minutes Maximum)

Videos longer than 5:00 are cut at 5:00. Show the working product, not only slides. 1080p, clear audio, captions recommended; unlisted YouTube, Vimeo or Drive link.

| Time        | Segment             | What to Show                                                                                                                                                                                                                                                                                                                                                            |
|-------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0:00 – 0:30 | Problem             | A breakdown costs ~\$2,400 on the road vs ~\$520 in the workshop; rules flood managers or miss slow failures; 3 fleets, 100K vehicles, 3 OEM formats                                                                                                                                                                                                                    |
| 0:30 – 1:00 | Solution            | "The list of vehicles to pull in this week, with the reason and the saving, plus sub-second critical alerts"; Overview with live KPIs                                                                                                                                                                                                                                   |
| 1:00 – 3:00 | Live Demo           | Three OEM formats, a bad record to the DLQ; overheat scenario → critical alert in < 1 s; vehicle 360 with coolant crossing the alert line; live map; predictions with reasons and \$; one-click work order; copilot proposal approved, injection refused; idle cost and driver safety; analyst sees 5 km cells, other tenant gets "not found"; audit verify and erasure |
| 3:00 – 4:15 | Under the Hood      | Architecture diagram; Grafana throughput, lag and latency; kill a stream processor and watch recovery; chaos and load results; Helm and Terraform; one EXPLAIN before/after                                                                                                                                                                                             |
| 4:15 – 5:00 | Impact & Next Steps | Precision 0.91 vs 0.41, \$8.3M vs \$4.3M in the back-test, alert p95 480 ms, API p95 135 ms; pilot plan; team                                                                                                                                                                                                                                                           |

- **Video Link:** [URL] (recording plan: docs/demo-script.md)

## 14. Repository Checklist

- **README:** Problem, architecture image, quick start, environment variables, test commands, known issues
- **One-Command Run:** docker compose up (or make run) starts the full stack with the data simulator
- **Structure:** Clear service folders, plus /docs, /infra and /tests
- **CI Pipeline:** Build, lint, all test suites and security scans on every push
- **Hygiene:** No secrets committed, .env.example provided, commits from all members
- **Final Tag:** v1.0-submission

| Item            | Status             | Where / note                                                                                                                |
|-----------------|--------------------|-----------------------------------------------------------------------------------------------------------------------------|
| README          | Done               | Problem, architecture image, quick start, env vars, test commands, known issues (README.md)                                 |
| One-command run | Done               | docker compose up -d --build: infra, migrations, 100K-vehicle seed, model, simulator, pipeline, API, web, monitoring        |
| Structure       | Done               | services/*, docs/, infra/, tests/, deploy/, contracts/, db/                                                                 |
| CI pipeline     | Done               | .github/workflows/ci.yml: build, lint, all test suites with coverage gates, security scans, SBOMs, IaC validation, e2e      |
| Hygiene         | Partly team action | No secrets committed (local-only defaults, gitleaks in CI), .env.example provided; commits from all members: team to ensure |
| Final tag       | Team action        | Create v1.0-submission on the final commit                                                                                  |

## 15. Conclusion

- Summary of key learnings
- Strengths of the solution
- Challenges tackled and solved

**Key learnings.** Load and chaos testing found real defects that unit tests could not: a query that was fast in isolation but 351 ms under RLS; MQTT messages silently lost to orphaned shared-subscription sessions; a simulator rate limiter that stalled; topics created with replication factor 1 in the production chart; a 45 s failover gap for critical alerts; security headers dropped by nginx. Measuring first and writing claims second kept the design honest.

**Strengths.** An end-to-end working product on a 100K-vehicle dataset; explainable predictions that beat today's rules at equal cost; database-enforced multi-tenancy and privacy; an AI assistant with real guardrails; one-command local run and a cloud-agnostic deployment path; evidence for every number.

**Challenges tackled.** Three incompatible OEM formats (declarative adapters); at-least-once delivery (Bloom dedup + deterministic IDs); security-barrier views vs performance (fenced LATERAL); rare-event ML (down-sampling with prior correction, time-split evaluation); tenant-safe AI (permission-scoped tools, approval workflow); limited build hardware (per-stage benchmarks and capacity modelling).

## 16. Declarations

- **Open-Source Components:** List with licences (an SBOM file in the repository is acceptable)

- **AI Tools Used:** Coding assistants or LLM APIs, and the parts of the work they were used for
- **Data:** Confirm all data is synthetic or public and contains no real personal data

**Open-source components** (full list with licences in docs/declarations.md; CycloneDX SBOM per image from CI):

| Area          | Components (licence)                                                                                                                                                                                                                                                          |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Data services | Apache Kafka, ClickHouse (Apache-2.0); PostgreSQL + pgvector (PostgreSQL Licence); Redis 7.4 local image (RSALv2/SSPLv1; production ElastiCache 7.1 BSD, Valkey drop-in); Mosquitto (EPL-2.0/EDL-1.0); nginx (BSD-2); Prometheus (Apache-2.0); Grafana (AGPL-3.0, unmodified) |
| Go            | franz-go, google/uuid, x/crypto (BSD-3); go-redis (BSD-2); pgx, go-json, fastjson (MIT); paho.mqtt (EPL-2.0/EDL-1.0); client_golang, jsonschema (Apache-2.0)                                                                                                                  |
| Python        | FastAPI, Pydantic, PyJWT, redis-py, anthropic SDK (MIT); Uvicorn, httpx, numpy, pandas, scikit-learn (BSD-3); asyncpg, pyarrow, bcrypt, prometheus-client (Apache-2.0); psycpg (LGPL-3.0)                                                                                     |
| JavaScript    | React, React Router, TanStack Query, Recharts, Tailwind, Vite (MIT); Leaflet (BSD-2); react-leaflet (Hippocratic-2.1); lucide-react, topojson-client, world-atlas (ISC); map tiles © OpenStreetMap contributors (ODbL)                                                        |

**AI tools used.** Claude Code (AI coding assistant) was used throughout, under the team's direction and review, for code, tests, infrastructure, documentation and this document. The optional runtime copilot uses whichever LLM the operator configures (any OpenAI-compatible provider, e.g. a free tier, or Claude); without one it answers with the offline planner.

**Data.** All data is synthetic, generated from a fixed seed by the project's simulator (tenants, 100,000 vehicles, 80,003 drivers with generated names and identifiers, telemetry, faults and service history). No real personal, vehicle or customer data is used. The fault knowledge base was written for the project from publicly documented OBD-II code meanings.

## 17. Appendix (if any)

- Additional diagrams or flowcharts
- Full test logs or large datasets used
- References or external sources

### Additional material in the repository:

- Diagrams: docs/architecture.md (C4, data flow, sequences), docs/erd.md, docs/diagrams/\*.mmd|png
- Design: docs/adr/0001-0006, docs/algorithms.md, docs/sql-optimization.md, docs/capacity.md, docs/security.md, docs/ai-ml.md, docs/observability.md

- Evidence: docs/evidence/ (k6 load and per-endpoint results, chaos report, EXPLAIN plans, pipeline benchmarks), docs/screenshots/
- API: docs/api/openapi.json; event contracts: contracts/\*.schema.json; OEM samples: contracts/oem-samples/

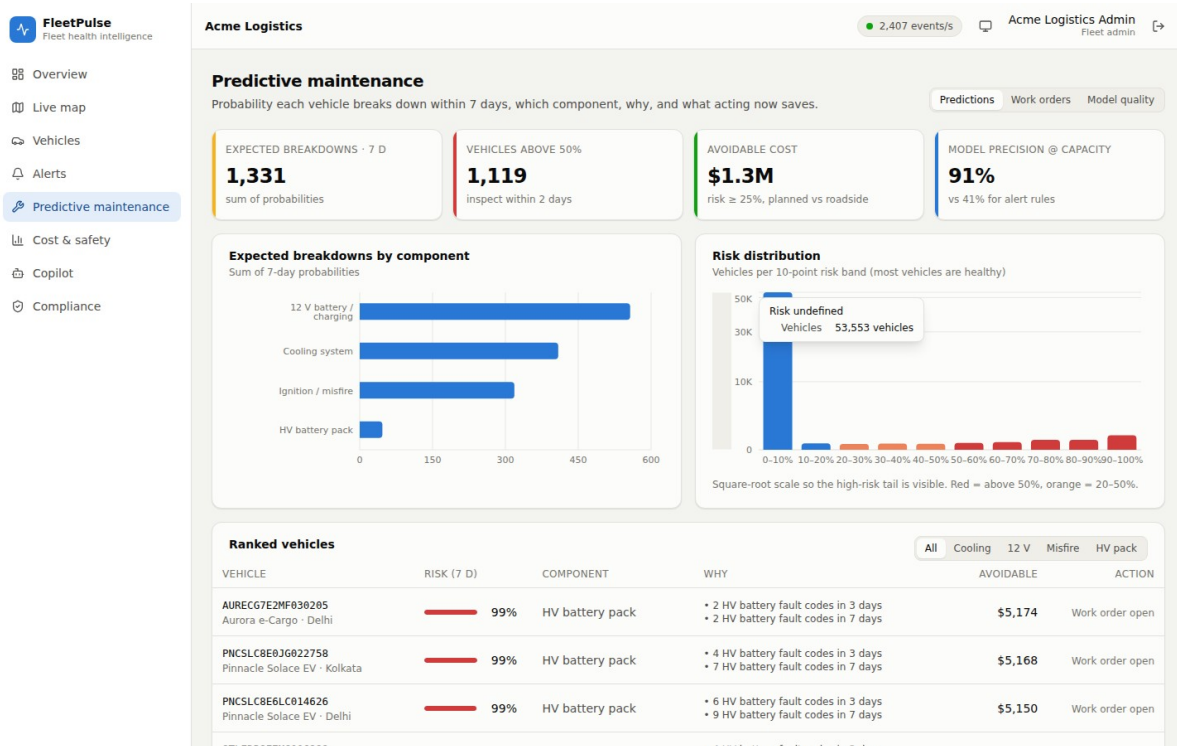


Figure 11. Predictive maintenance: ranked vehicles with component, reasons and savings.

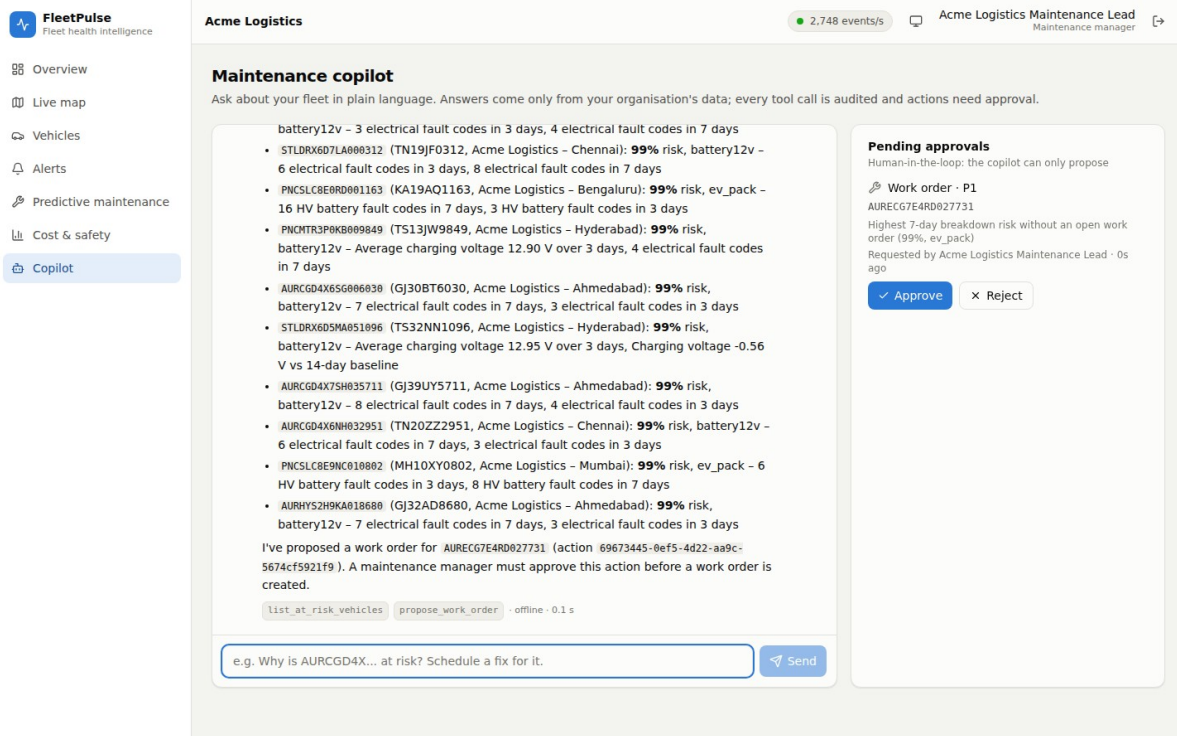


Figure 12. Copilot: ranking, then a work-order proposal awaiting approval.


**FleetPulse**  
Fleet health intelligence

**Acme Logistics**
2,746 events/s
Acme Logistics Admin
Fleet admin

Overview  
 Live map  
 Vehicles  
 Alerts  
 Predictive maintenance  
 Cost & safety  
 Copilot  
**Compliance**

### Compliance

Tamper-evident audit trail of every data access and AI action; data-subject erasure (GDPR Art. 17 / DPDP Act s.12).

Audit trail
Right to erasure

Everyone
People
AI copilot
Verify integrity

| #      | TIME           | ACTOR            | ACTION                              | RESOURCE    | OUTCOME | HASH            |
|--------|----------------|------------------|-------------------------------------|-------------|---------|-----------------|
| 164114 | 30 Sept, 20:21 | USER<br>da8324c9 | GET /api/v1/auth/me                 | auth        | ALLOW   | 968eda48a8ab... |
| 164112 | 30 Sept, 20:21 | USER<br>da8324c9 | POST /api/v1/live/ws-ticket         | live        | ALLOW   | d521a2e619a7... |
| 164110 | 30 Sept, 20:21 | USER<br>da8324c9 | GET /api/v1/copilot/actions         | copilot     | ALLOW   | f895b483876...  |
| 164108 | 30 Sept, 20:21 | USER<br>da8324c9 | GET /api/v1/auth/me                 | auth        | ALLOW   | e8aea881362f... |
| 164106 | 30 Sept, 20:21 | USER<br>da8324c9 | GET /api/v1/analytics/idle-cost     | analytics   | ALLOW   | 8f84fc12f8e8... |
| 164104 | 30 Sept, 20:21 | USER<br>da8324c9 | GET /api/v1/analytics/driver-safety | analytics   | ALLOW   | 4b3423f7854...  |
| 164102 | 30 Sept, 20:21 | USER<br>da8324c9 | GET /api/v1/analytics/fleet-hourly  | analytics   | ALLOW   | 8fc888f6db47... |
| 164100 | 30 Sept, 20:21 | USER<br>da8324c9 | POST /api/v1/live/ws-ticket         | live        | ALLOW   | cdd29a49c1da... |
| 164098 | 30 Sept, 20:21 | USER<br>da8324c9 | GET /api/v1/auth/me                 | auth        | ALLOW   | 653e78a2ecbe... |
| 164096 | 30 Sept, 20:21 | USER<br>da8324c9 | GET /api/v1/maintenance/summary     | maintenance | ALLOW   | 77d31868222a... |
| 164094 | 30 Sept, 20:21 | USER<br>da8324c9 | GET /api/v1/maintenance/risk        | maintenance | ALLOW   | 497285c193b3... |

Figure 13. Compliance: tamper-evident audit trail with integrity verification.

**References:** Apache Kafka, ClickHouse, PostgreSQL (row-level security, pgvector) and Kubernetes documentation; ISO 3779 (VIN); SAE J2012 / OBD-II diagnostic trouble codes; Bloom (1970) and Kirsch & Mitzenmacher (2006) on Bloom filters; Cormode & Muthukrishnan (2005) Count-Min Sketch; GDPR Art. 17; India DPDP Act 2023; OWASP ASVS and Top 10.